Studio Nx

Backend(FastAPI/Python) 58 — to_item() · list_projects()

2026-10-12

Backend(FastAPI/Python) 58 — `to_item()` · `list_projects()` (내 프로젝트 목록)

항목	내용
파일	`studionx/backend/app/services/project_service.py`
작업	P1-13 프로젝트 API: 목록, 검색, 정렬, 페이지 (PRD 01 FR-123, FR-130)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	57 `get_membership` · `require_role` · 59 `create_project`

1. 하는 일

- **`to_item`**: DB의 `Project` 한 행을 화면이 쓰는 모양인 `ProjectItem`으로 바꿉니다. 프로젝트 칸에 더해서 세 칸을 계산해 넣습니다.
- **`list_projects`**: 내가 멤버인 프로젝트를 검색하고 정렬한 뒤 페이지 단위로 잘라서 `{items, total, page, size}`로 돌려줍니다.

계산하는 칸	뜻	화면 (myProject.html)
`my_role`	이 프로젝트에서 내 역할 (OWNER, EDITOR, VIEWER)	카드의 역할 표시에 쓰이고, 삭제 버튼은 OWNER에게만 열립니다
`member_count`	멤버 수	카드의 멤버 수
`opened`	**내가** 시퀀서에서 연 적이 있는지 (`project_sessions` 행이 있는지)	한 번도 열지 않았으면 Not opened를 표시합니다 (FR-130)

요청 예	결과
`GET /projects`	최근 수정순으로 1쪽 10개
`?sort=title`	이름순입니다 (대소문자 무시: `alpha`, `Beta`, `Gamma`)
`?sort=created`	만든 순서입니다 (최근 것 먼저)
`?query=alp`	제목에 `alp`가 들어간 프로젝트입니다 (대소문자 무시)
`?query=%`	제목에 **글자 `%`**가 들어간 프로젝트만 나옵니다 (모든 프로젝트가 나오지 않습니다)
`?page=3` (2쪽까지만 있음)	`items`는 빈 목록이고, `total`은 그대로입니다

2. 언제 쓰나

호출하는 곳	쓰는 것
`GET /api/v1/projects` 라우터 (62)	`list_projects`
`GET`, `POST`, `PATCH /projects/...` 응답 (62, 59, 60)	`to_item(db, project, member)`: 세 칸을 직접 DB에 물어봅니다
`list_projects` 안	`to_item(..., member_count, opened)`: 미리 모아서 계산한 값을 넘깁니다
화면	myProject.html의 `loadProjects`, `renderProjects` (P1-14)

`project_sessions`는 시퀀서를 열 때마다 생기는 작업 기록입니다 (기록은 Phase 2부터 생깁니다). 지금은 행이 없으므로 모든 프로젝트가 `opened=false`로 나옵니다.

3. 코드

import (파일 맨 위)

```
from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Project, ProjectMember, ProjectSession, User
from app.schemas.project import (
    ProjectCreateRequest,
    ProjectItem,
    ProjectListResponse,
    ProjectSort,
    ProjectUpdateRequest,
)
```

`from sqlalchemy import func, select` 줄과 `ProjectSession`을 이번에 추가했습니다. 두 함수는 `func`, `select`, `Session`, `Project`, `ProjectMember`, `ProjectSession`, `User`, `ProjectItem`, `ProjectListResponse`, `ProjectSort`를 사용합니다.

함수 (도움 함수 `_escape_like`와 정렬 표 `SORT_ORDER` 포함)

```
def to_item(
    db: Session,
    project: Project,
    member: ProjectMember,
    member_count: int | None = None,
    opened: bool | None = None,
) -> ProjectItem:
    if member_count is None:
        member_count = db.scalar(
            select(func.count())
            .select_from(ProjectMember)
            .where(ProjectMember.project_id == project.project_id)
        )
    if opened is None:
        session_id = db.scalar(
            select(ProjectSession.session_id)
            .where(
                ProjectSession.project_id == project.project_id,
                ProjectSession.user_id == member.user_id,
            )
            .limit(1)
        )
        opened = session_id is not None
    return ProjectItem(
        project_id=project.project_id,
        title=project.title,
        description=project.description,
        bpm=project.bpm,
```

```
        time_sig_num=project.time_sig_num,
        time_sig_den=project.time_sig_den,
        music_key=project.music_key,
        created_at=project.created_at,
        updated_at=project.updated_at,
        my_role=member.role,
        member_count=member_count,
        opened=opened,
    )


def _escape_like(text: str) -> str:
    return text.replace("\\", "\\\\").replace("%", "\\%").replace("_", "\\_")


SORT_ORDER = {
    ProjectSort.UPDATED: Project.updated_at.desc(),
    ProjectSort.TITLE: Project.title.asc(),
    ProjectSort.CREATED: Project.created_at.desc(),
}


def list_projects(
    db: Session,
    user: User,
    query: str,
    sort: ProjectSort,
    page: int,
    size: int,
) -> ProjectListResponse:
    conditions = [ProjectMember.user_id == user.user_id, Project.deleted_at.is_(None)]
    if query:
        conditions.append(Project.title.ilike(f"%{_escape_like(query)}%", escape="\\"))

    total = db.scalar(
        select(func.count())
        .select_from(Project)
        .join(ProjectMember, ProjectMember.project_id == Project.project_id)
        .where(*conditions)
    )
    rows = db.execute(
        select(Project, ProjectMember)
        .join(ProjectMember, ProjectMember.project_id == Project.project_id)
        .where(*conditions)
        .order_by(SORT_ORDER[sort], Project.project_id.desc())
        .offset((page - 1) * size)
        .limit(size)
    ).all()

    # 이 페이지 프로젝트들의 멤버 수 · 내가 연 적 있는지를 쿼리 두 번으로 모아 계산
    ids = [project.project_id for project, _ in rows]
    counts = {}
    opened_ids = set()
    if ids:
        counts = dict(
            db.execute(
                select(ProjectMember.project_id, func.count())
```

```
                .where(ProjectMember.project_id.in_(ids))
                .group_by(ProjectMember.project_id)
            ).all()
        )
        opened_ids = set(
            db.scalars(
                select(ProjectSession.project_id).where(
                    ProjectSession.project_id.in_(ids), ProjectSession.user_id == user.user_id
                )
            )
        )

    items = [
        to_item(db, project, member, counts[project.project_id], project.project_id in opened_ids)
        for project, member in rows
    ]
    return ProjectListResponse(items=items, total=total, page=page, size=size)
```

문법 풀이

import 문법	뜻
`from sqlalchemy import func, select`	`select(...)`는 SELECT 문을 만들고, `func.count()`는 SQL의 `COUNT(*)`가 됩니다

식	뜻
`member_count: int | None = None`	값을 넘기지 않으면 None이 되고, 함수 안에서 DB에 물어봅니다
`db.scalar(select(...))`	결과의 **첫 행 첫 칸** 하나만 가져옵니다 (개수나 번호 하나)
`.limit(1)` + `is not None`	행이 **하나라도 있는지**만 확인합니다 (전부 세지 않습니다)
`SORT_ORDER[sort]`	정렬 종류를 `ORDER BY` 식으로 바꿉니다. `if/elif` 세 줄 대신 표를 씁니다
`.where(*conditions)`	목록에 담긴 조건을 모두 AND로 묶습니다 (`*`는 목록을 풀어서 인자 여러 개로 넘깁니다)
`Project.title.ilike(..., escape="\\")`	대소문자를 무시하는 `LIKE`입니다. 검색어 안에서 `\` 뒤에 오는 글자는 특수 문자가 아니라 글자 그대로 취급됩니다
`.offset((page - 1) * size).limit(size)`	3쪽, 10개씩이라면 앞의 20개를 건너뛰고 10개를 가져옵니다
`db.execute(select(Project, ProjectMember)...).all()`	행마다 `(project, member)` 두 객체를 가져옵니다
`.in_(ids)`, `.group_by(...)`	`WHERE project_id IN (...)`과, 프로젝트별로 묶어서 세는 조건입니다
`dict(rows)`	`[(5, 2), (8, 1)]`을 `{5: 2, 8: 1}`(프로젝트 번호 → 멤버 수)로 바꿉니다
`project.project_id in opened_ids`	집합(set) 안에 있는지 확인해서 True 또는 False를 돌려줍니다

4. 흐름

1. 조건을 만듭니다: 내가 멤버이고, 삭제되지 않았으며, 검색어가 있으면 제목 LIKE 조건을 더합니다.
2. 같은 조건으로 **전체 개수**(`total`)를 셉니다.
3. 정렬하고, 값이 같으면 `project_id`가 큰 것을 먼저 둡니다. 그다음 해당 페이지만 잘라서 `(project, member)`를 가져옵니다.

4. 이 페이지의 프로젝트 번호들로 **쿼리를 두 번** 보냅니다: 멤버 수(묶어서 세기)와 내가 연 프로젝트 번호입니다.
5. 행마다 `to_item`에 계산한 값을 넘겨 `ProjectItem`을 만들고 응답합니다.

목록을 한 번 받는 데 쿼리는 **4번**(개수, 목록, 멤버 수, 연 적 있음) 나갑니다. 프로젝트가 10개이든 50개이든 같습니다.

5. 왜 이렇게

선택	이유
멤버 수와 opened를 **모아서** 계산	행마다 물으면 10개짜리 목록에 쿼리가 20번 더 나갑니다 (N+1 문제). 템플릿이 권장한 방식입니다
`to_item`은 값을 넘기지 않으면 직접 물어봄	상세, 만들기, 수정 응답은 프로젝트가 하나뿐이라 모을 필요가 없습니다. 한 함수를 두 곳에서 같이 씁니다
정렬 뒤에 `project_id DESC`를 붙임	수정 시각이 같은 프로젝트들의 순서가 쿼리마다 바뀌면, 2쪽에 1쪽 프로젝트가 또 나오거나 빠질 수 있습니다
검색어의 `%`, `_`, `\`를 escape	`LIKE`에서 `%`는 아무 글자 여러 개, `_`는 아무 글자 하나를 뜻합니다. 그대로 두면 `%` 검색이 모든 프로젝트를 돌려줍니다 (템플릿 권장)
남이 만들었지만 내가 멤버인 프로젝트도 포함	FR-123의 "내가 참여한 프로젝트"에 해당합니다. 역할은 `my_role`로 구분합니다 (Phase 6 초대 기능 대비)
`opened`는 **내** 세션만 확인	다른 멤버가 열었어도 내가 아직 열지 않았다면 Not opened입니다 (FR-130은 사람마다 따로 판단합니다)
없는 페이지는 빈 목록 (404가 아님)	마지막 프로젝트를 지운 뒤 그 페이지에 머물러 있어도, 화면이 오류 대신 "없음"을 보여 줄 수 있습니다

6. 시험 (`tests/test_project_service.py`: 13개를 추가해서 모두 25개)

시험	확인한 것
`to_item` 계산	멤버가 2명이면 `member_count`는 2이고, 남이 연 세션은 무시하며, 내 세션이 생기면 `opened`가 True가 됩니다
`to_item`에 값 넘기기	넘긴 7과 True를 그대로 쓰고, 프로젝트 칸을 복사합니다
정렬 3가지	updated, title(대소문자 무시), created 순서가 맞습니다
같은 값	수정 시각과 생성 시각이 같으면 나중에 만든 것이 먼저 나옵니다
페이지	2개씩 나누면 1쪽 2개, 2쪽 1개, 3쪽은 빈 목록이고, `total`은 항상 3입니다
검색	`ALP`로 `alpha`를 찾습니다 (대소문자 무시)
특수 문자	`%`로는 `100% Mix`만, `_`로는 `my_song`만 찾습니다
범위	남의 것과 삭제된 것은 빠지고, 멤버로 참여한 남의 프로젝트는 EDITOR, 멤버 2명으로 포함됩니다
opened 목록	내가 연 프로젝트만 True입니다
빈 목록	프로젝트가 없으면 `items=[]`, `total=0`입니다

전체 시험 463개가 통과했고, `ruff` 검사도 통과했습니다.